

Лабораторна робота №1.

Локальний сервінг і надійний typed-клієнт

25 серпня 2026 р.

Мета

Розгорнути дві локальні мовні моделі, виміряти їхню поведінку під навантаженням і побудувати навколо них сервіс, який перетворює недовірений текст на валідований типізований об'єкт або на контрольовану відмову — і жодного разу не повертає неперевірений результат як успішний.

Робота виконується **бригадою з трьох осіб**. Лекції 1–5 і 9.

Усе безкоштовне. Жодного платного API: моделі запускаються локально через Ollama або llama.cpp, усі бібліотеки відкриті. Якщо в когось із бригади немає відеокарти, працює CPU-профіль: модель 0,5–1,5B і текст 2k.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — сервінг і вимірювання.** Розгортає дві моделі, знімає TTFT, TROT і пропускну здатність, рахує бюджет пам'яті за формулами і звіряє з фактом.
- **Учасник В — контракт і розбір.** Описує схему результату, вмикає обмежене декодування, реалізує розбір із однією спробою відновлення і класифікацію помилок за чотирма класами.
- **Учасник С — надійність і перевірка.** Реалізує політику повторів, таймаути, бюджет часу та ідемпотентність; складає набір випадків і пише тести.

Інтеграційний крок виконують разом: клієнт учасника В і механізми учасника С загортаються у HTTP-сервіс поверх сервінгу учасника А, після чого набір учасника С проганяється на обох моделях учасника А.

Теоретичні відомості

Мовна модель повертає розподіл над продовженнями, а не значення функції. Та сама пара «інструкція + запит» може дати різні виходи, і причина не лише в семплінгу: на результат впливають версія моделі, склад і порядок контексту, а на GPU ще й пакування запитів разом із неасоціативністю операцій з рухомою комою. Тому текст відповіді не є інтерфейсом, а межа між моделлю і кодом описується контрактом.

- **Дві фази запиту мають різну природу.** Префіл обробляє весь запит одразу і впирається в обчислення; декод породжує токени по одному і впирається в пропускну здатність пам'яті, бо на кожен токен перераховуються всі ваги. Звідси дві різні метрики: час до першого токена (TTFT) залежить від довжини запиту, час на наступний токен (TPOT) — від пам'яті й розміру моделі. Наскрізна затримка складається як $T = \text{TTFT} + \text{TPOT} \cdot N_{\text{out}}$.

- **Пам'ять ділиться на ваги і KV-кеш:**

$$S_{KV} = 2 \cdot L \cdot n_{kv} \cdot d_h \cdot b \cdot s \cdot B$$

де L — шарів, n_{kv} — KV-голів, d_h — вимір голови, b — байтів на число, s — довжина, B — батч. Саме залишок пам'яті під кеш, а не заявлена довжина контексту, задає кількість одночасних сеансів.

- **Гарантій формату є три рівні, і вони не взаємозамінні.** Інструкція в промпті «відповідай у JSON» не гарантує нічого. Валідація після генерації гарантує, що некоректний результат не пройде далі, ціною повторних викликів. Обмежене декодування будує граматику допустимих продовжень і забороняє токени, що виводять за межі схеми; синтаксична валідність тоді гарантована процедурою генерації. Локально цю роль виконують GBNF-граматики в `llama.cpp` або параметр `format` в Ollama.
- **Схема описує форму, а не зміст.** Заборона зайвих полів (`additionalProperties: false`) не косметична: без неї модель домальовує поля, яких немає в контракті, і вони мовчки проходять далі. Але навіть строга схема гарантує лише те, що поле існує і має потрібний тип. Доменні правила перевіряє код.
- **Чотири класи помилок вимагають чотирьох різних реакцій.** *Транспортна* каже, що виклик не дійшов, — її повторюють із затримкою. *Схемна* означає, що відповідь не лягла в контракт, — повторюють не більше двох разів, передаючи моделі текст помилки валідатора. *Доменна* означає, що відповідь валідна за формою і неприйнятна за правилами задачі, — повтор не допоможе. *Відмова моделі* є законною відповіддю, а не збоєм.

- **Ідемпотентність робить повтор безпечним.** Ключ операції обчислюється до виклику з тих даних, що визначають операцію, і зберігається разом із результатом. Без ключа будь-який ретрай на дії зі станом подвоює наслідок.

Корисні посилання для ознайомлення:

- [Каталог моделей Ollama](#) і [FAQ](#) зі змінними оточення;
- [Pydantic: JSON Schema](#) та [Strict Mode](#);
- llama.cpp: [GBNF Grammars](#);
- М. Brooker, [Exponential Backoff And Jitter](#);
- [FastAPI](#) для сервісної обгортки та [Locust](#) для навантажувального тесту.

Порядок виконання

Позначка [A], [B], [C] вказує відповідального за крок; **[разом]** — крок виконує вся бригада.

1. **[разом] Домовитися про контракт.** До написання коду бригада узгоджує предметну область і структуру результату: не менше п'яти полів, серед них числове з діапазоном, перелічуване і необов'язкове. Контракт фіксується у файлі `contract.md` і далі не змінюється без спільного рішення.
Перевірка: усі троє однаково відповідають на питання, що система повертає при відсутності відповіді.
2. **[A] Розгорнути дві моделі** різного розміру через Ollama або llama.cpp — наприклад, 1B і 7–8B у 4-бітному кванті. Явно задати довжину контексту й кількість паралельних запитів.
Перевірка: ollama ps показує, що модель повністю на GPU (або свідомо на CPU). Частковий перенос шарів — найчастіша тиха причина того, що всі подальші числа неправильні.
3. **[A] Виміряти поведінку:** TTFT, TROT і пропускну здатність при конкурентності 1, 2, 4 (для GPU — до 8) на однаковому наборі з 20 запитів. Побудувати таблицю p50 і p95 для обох моделей. Окремо поррахувати за формулами обсяг ваг і KV-кешу для двох довжин контексту і порівняти з фактичним споживанням.
Перевірка: перед кожним заміром — розігрів; розбіжність розрахунку з фактом пояснена, а не списана на похибку.

4. **[В] Описати контракт кодом:** модель `Pydantic` за домовленою структурою, заборона додаткових полів, доменний валідатор. Вивести `JSON Schema` і додати до звіту.
Перевірка: об'єкт із зайвим полем валідацію не проходить.
5. **[В] Увімкнути обмежене декодування** — параметр `format` в `Ollama` або `GBNF-граматика` в `llama.cpp` — і показати різницю: скільки невалідних відповідей дає та сама модель без обмеження і з ним на 30 запитах.
6. **[В] Реалізувати розбір і класифікацію помилок:** рівно одна спроба відновлення з текстом помилки валідатора; чотири класи помилок, для кожного типізований об'єкт відмови з полями клас, повідомлення, чи доречний повтор.
Перевірка: у логах видно, що друга спроба отримала саме текст помилки, а не той самий промпт.
7. **[С] Реалізувати механізми надійності:** повтор з експоненційним зростанням затримки та джитером, таймаут на виклик, бюджет часу на запит, запобіжник після N поспіль невдач і резервний перехід на меншу модель.
Перевірка: штучно сповільнити модель і показати в логах, що обробку зупинив саме бюджет, а не виняток.
8. **[С] Реалізувати ідемпотентність:** ключ операції рахується до виклику, результати зберігаються у `SQLite`. Повторний запит із тим самим ключем не створює другого запису.
9. **[С] Скласти набір і тести:** не менш ніж 40 випадків — коректні, з відсутнім полем, з полем неправильного типу, із зайвим полем, з порушенням доменного правила, з обірваним `JSON`, з відмовою моделі, з таймаутом. Не менше 15 тестів `pytest`, усі проходять у режимі `stub` без мережі.
Перевірка: якщо для тестів потрібна жива модель — контракт спроектовано неправильно.
10. **[разом] Зібрати сервіс:** загорнути клієнта у `HTTP`-сервіс на `FastAPI` з одним ендпоінтом, який приймає запит і повертає валідований об'єкт або типізовану відмову. Прогнати набір учасника `C` на обох моделях учасника `A`.
11. **[разом] Навантажувальний тест:** `Locust` або власний скрипт, 3 рівні конкурентності. Показати, як поведуться таймаут, запобіжник і резервна модель під навантаженням.
12. **[разом] Звіт:** порівняльна таблиця двох моделей (якість на наборі, затримки, пам'ять), таблиця чотирьох класів помилок із частотою кожного, висновок до 250 слів про те, яку модель ви обрали б для навчального відділу і чому.

Типова помилка порядку: писати тести після коду. Набір випадків із кроку 9 корисніше скласти одразу після кроку 1 — тоді контракт перевіряється на реальних поламаних даних.

Вимоги до задачі та захисту

1. Репозиторій з `README`, у якому одна команда піднімає сервіс і одна запускає тести. У `README` — таблиця «хто що робив» із посиланнями на коміти.
2. Історія комітів має показувати внесок кожного учасника. Робота, зроблена одним за трьох, оцінюється як робота одного.
3. На захисті кожен учасник відповідає за **свою** підсистему і за один спільний крок. Питання ставляться персонально.
4. Спільний висновок захищає бригада: чому обрано саме цю модель і які межі має ваше рішення.

Контрольні запитання

1. Чому температура 0 не гарантує однакового виходу на однаковому вході?
2. Чому декод упирається в пропускну здатність пам'яті, а не в обчислення?
3. Що саме гарантує обмежене декодування і чого воно не гарантує?
4. Навіщо в схемі `additionalProperties: false`, якщо зайві поля можна ігнорувати?
5. Чим доменна помилка відрізняється від помилки схеми і чому реакція на них різна?
6. У яких випадках повторний виклик робить гірше, ніж відмова?
7. Що станеться, якщо у політиці повтору немає випадкової складової?
8. Як обчислити ключ ідемпотентності для запиту з довільним текстом користувача?
9. Чому `r95` зростає нелінійно за високого завантаження?
10. Ваша менша модель дала вищу якість на наборі, ніж більша. Які пояснення можливі й як їх перевірити?

Література

1. Willard B., Louf R. [Efficient Guided Generation for Large Language Models](#). arXiv:2307.09702, 2023.
2. Kwon W. та ін. [Efficient Memory Management for LLM Serving with PagedAttention](#). SOSP, 2023.
3. Xiao T., Zhu J. [Foundations of Large Language Models](#). arXiv:2501.09223, 2025. Розділ 5.
4. Jurafsky D., Martin J. H. [Speech and Language Processing](#), 3rd ed. draft, 2026. Розділ 7.